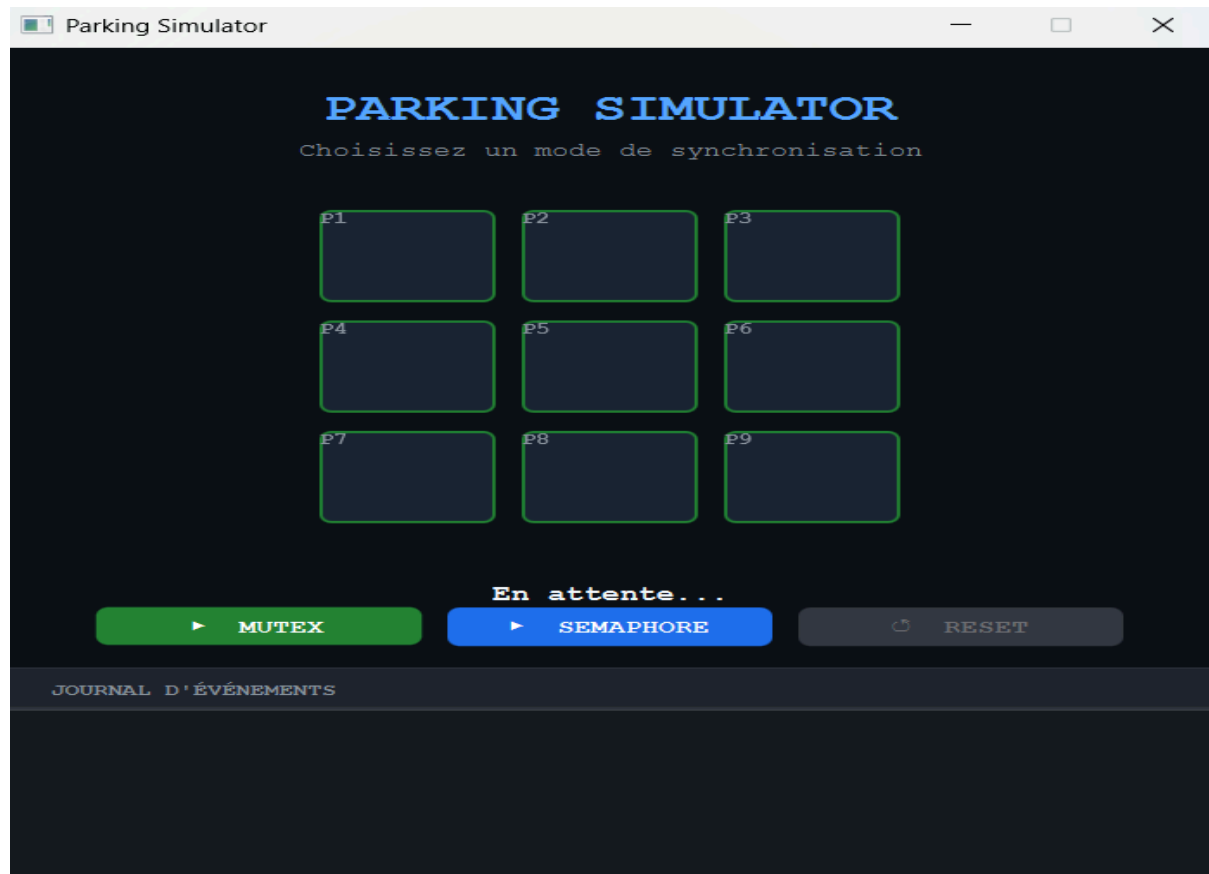


PARKING SIMULATOR

Guide d'Utilisation et compte rendu

Simulation du Problème de Parking avec Mutex & Sémaphore

Version 1.0-SNAPSHOT — Java 21 + JavaFX + Maven



Interface principale du Parking Simulator

Projet	Simulateur de Parking – Mini Projet SysAdm
Date	14 mai 2026
Technologies	Java 21, JavaFX, Apache Maven 3.9.15, Windows 11
Dépôt GitHub	https://github.com/youssef-opencodes/ParkingSimulator

1. Introduction

Le **Parking Simulator** est une application Java/JavaFX qui illustre la gestion concurrente de ressources partagées dans les systèmes d'exploitation. Il simule des voitures (threads) qui tentent de se garer dans un parking à places limitées, en utilisant deux mécanismes de synchronisation : le **Mutex** et le **Sémaphore**.

Ce guide explique comment installer, compiler, exécuter et interpréter les résultats de la simulation.

2. Prérequis

Logiciels requis :

Outil	Version / Remarque
Java JDK 21	Oracle JDK 21.0.9 (LTS) — <code>java -version</code> doit afficher 21.x
Apache Maven 3.9+	Maven 3.9.15 — <code>mvn -version</code> doit afficher 3.9.x
Git	Pour cloner le dépôt depuis GitHub
Windows 11 / Linux / macOS	Compatible toutes plateformes Java

Vérification de Java 21 : `jav`

```
PS C:\Users\smino\OneDrive\Desktop\khammal\SysAdm> java -version
java version "21.0.9" 2025-10-21 LTS
Java(TM) SE Runtime Environment (build 21.0.9+7-LTS-338)
Java HotSpot(TM) 64-Bit Server VM (build 21.0.9+7-LTS-338, mixed mode, sharing)
PS C:\Users\smino\OneDrive\Desktop\khammal\SysAdm> █
```

`a -version`

Vérification de Maven 3.9.15 : `m`

```
PS C:\Users\smino\OneDrive\Desktop\khammal\SysAdm> mvn -version
Apache Maven 3.9.15 (98b2cdbcdb5f1ac8781f537ea9accaed7922349)
Maven home: C:\Program Files\apache-maven-3.9.15
Java version: 21.0.9, vendor: Oracle Corporation, runtime: C:\Program Files\Java\jdk-21
Default locale: fr_FR, platform encoding: UTF-8
OS name: "windows 11", version: "10.0", arch: "amd64", family: "windows"
PS C:\Users\smino\OneDrive\Desktop\khammal\SysAdm> █
```

`vn -version`

3. Installation & Récupération du code source

3.1 Cloner le dépôt Git

Ouvrez un terminal (PowerShell, Git Bash ou terminal Linux) et exécutez :

```
git clone https://github.com/MrPee-Agent/SysAdm.git
cd SysAdm
```

3.2 Mettre à jour le code (git pull)

Si le dépôt est déjà cloné, synchronisez-le avec la dernière version :

```
git pull
```

Résultat de git pull — 7 fic

```
HP@DESKTOP-T7EGC60 MINGW64 ~/Desktop/sysAdm (main)
$ ls
README.md  docs/  makefile  pom.xml  src/  target/

HP@DESKTOP-T7EGC60 MINGW64 ~/Desktop/sysAdm (main)
$ git pull
remote: Enumerating objects: 83, done.
remote: Counting objects: 100% (82/82), done.
remote: Compressing objects: 100% (32/32), done.
remote: Total 66 (delta 19), reused 64 (delta 17), pack-reused 0 (from 0)
Unpacking objects: 100% (66/66), 14.47 KiB | 54.00 KiB/s, done.
From https://github.com/MrPee-Agent/SysAdm
   566ed20..a851bb3  main -> origin/main
* [new branch]      interface-dynamique -> origin/interface-dynamique
Updating 566ed20..a851bb3
Fast-forward
 .gitignore                  | 3 -
 docs/Guide_Utilisation.md  | 0
 logs/parking.log           | 0
 makefile                   | 26 ++++++++
 .../com/example/ParkingSimulator/core/Parking.java | 13 ++++
 .../example/ParkingSimulator/sync/MutexSync.java   | 10 +++-
 .../example/ParkingSimulator/utills/LoggerUtil.java | 70 ++++++++
++
7 files changed, 118 insertions(+), 4 deletions(-)
create mode 100644 docs/Guide_Utilisation.md
create mode 100644 logs/parking.log
create mode 100644 src/main/java/com/example/ParkingSimulator/utills/LoggerUtil.java
```

hiers modifiés, nouvelles branches détectées

3.3 Structure du projet

Après clonage, le répertoire contient les fichiers suivants :

- **README.md** — Documentation rapide du projet
- **pom.xml** — Fichier de configuration Maven (dépendances JavaFX, plugins)
- **makefile** — Raccourcis de compilation et d'exécution
- **src/** — Code source Java (Parking.java, MutexSync.java, LoggerUtil.java, UI...)
- **docs/** — Documentation technique et Utilisation et compte rendu
- **logs/** — Fichier parking.log généré après chaque simulation
- **target/** — Répertoire de build (généré par Maven)

4. Compilation du projet

Placez-vous à la racine du projet (répertoire contenant **pom.xml**) et lancez :

```
mvn clean compile
```

Cette commande nettoie le répertoire *target/* précédent, recompile les 11 fichiers source Java et copie les ressources. En cas de succès, Maven affiche **BUILD SUCCESS**.

Résultat de mvn clean co

```
PS C:\Users\smino\OneDrive\Desktop\khammal\SysAdm> mvn clean compile
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:ParkingSimulator >-----
[INFO] Building ParkingSimulator 1.0-SNAPSHOT
[INFO]    from pom.xml
[INFO] -----[ jar ]-----
[WARNING] 6 problems were encountered while building the effective model for org.openjfx:javafx-controls:jar:21.0.1 during dependency co
n step for project (use -X to see details)
[INFO]
[INFO] --- clean:3.2.0:clean (default-clean) @ ParkingSimulator ---
[INFO] Deleting C:\Users\smino\OneDrive\Desktop\khammal\SysAdm\target
[INFO]
[INFO] --- resources:3.4.0:resources (default-resources) @ ParkingSimulator ---
[INFO] Copying 2 resources from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.11.0:compile (default-compile) @ ParkingSimulator ---
[INFO] Changes detected - recompiling the module! :source
[INFO] Compiling 11 source files with javac [debug release 21 module-path] to target/classes
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time:  2.411 s
[INFO] Finished at: 2026-05-14T20:30:59+01:00
[INFO]
PS C:\Users\smino\OneDrive\Desktop\khammal\SysAdm> 
```

mpile — BUILD SUCCESS en 2.411s

■ **Note :** Les avertissements (WARNING) concernant org.openjfx:javafx-controls sont bénins et n'empêchent pas la compilation.

5. Exécution de la simulation

5.1 Lancer l'application

Depuis la racine du projet, exécutez :

```
mvn javafx:run
```

Lancement avec mvn ja

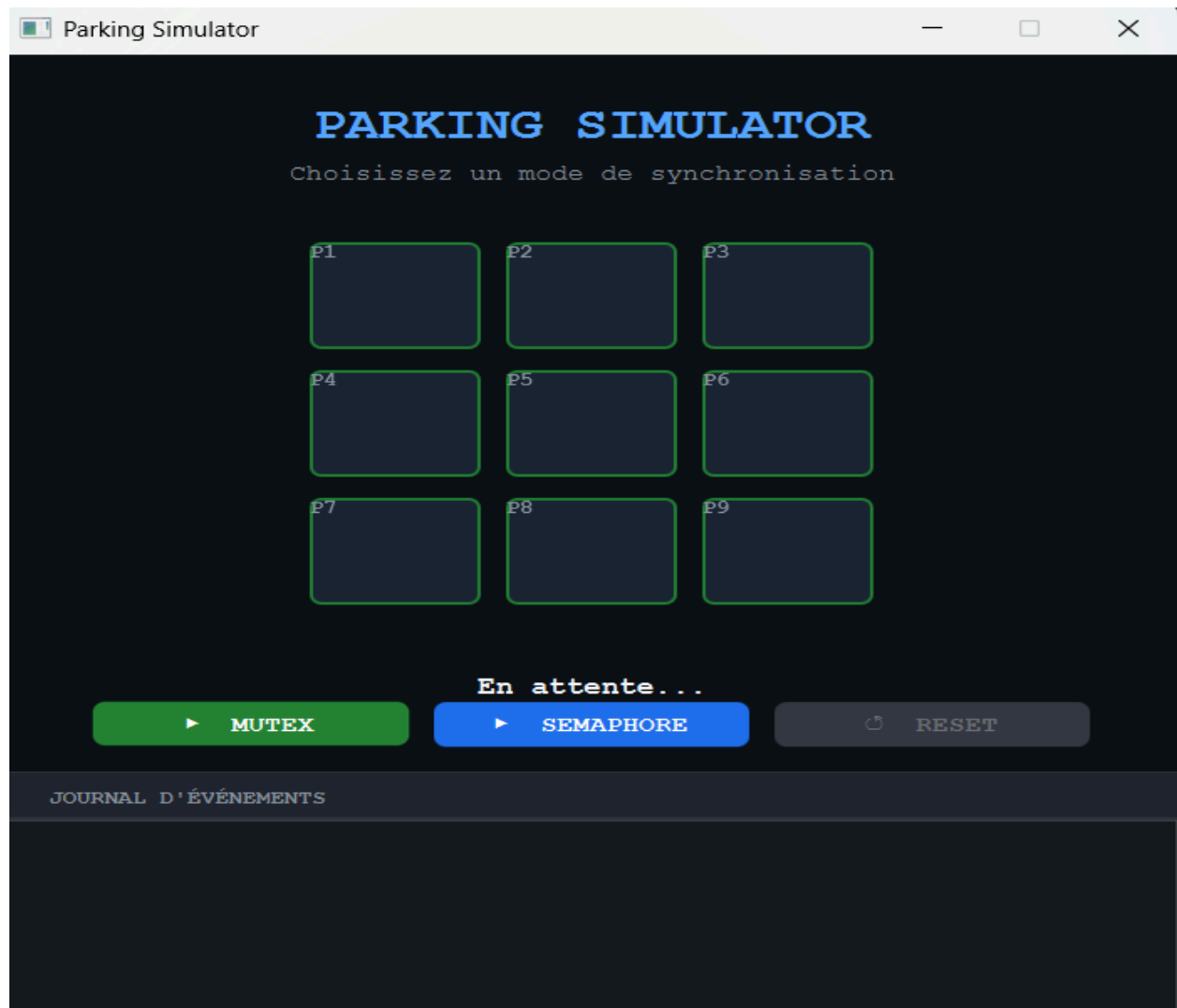
```
PS C:\Users\smino\OneDrive\Desktop\khammal\SysAdm> mvn javafx:run
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:ParkingSimulator >-----
[INFO] Building ParkingSimulator 1.0-SNAPSHOT
[INFO]    from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] >>> javafx:0.0.8:run (default-cli) > process-classes @ ParkingSimulator >>>
[WARNING] 6 problems were encountered while building the effective model for org.openjfx:javafx-controls:jar:21.0.1 during dependency collectio
n step for project (use -X to see details)
[INFO]
[INFO] --- resources:3.4.0:resources (default-resources) @ ParkingSimulator ---
[INFO] Copying 2 resources from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.11.0:compile (default-compile) @ ParkingSimulator ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] <<< javafx:0.0.8:run (default-cli) < process-classes @ ParkingSimulator <<<
[INFO]
[INFO]
[INFO] --- javafx:0.0.8:run (default-cli) @ ParkingSimulator ---
[WARNING] Module name not found in <mainClass>. Module name will be assumed from module-info.java
[INFO]

```

vafx:run — copie des ressources et exécution

5.2 Interface principale

Au démarrage, l'interface affiche un parking de **9 places** (P1 à P9), toutes disponibles (fond sombre avec bordure verte). Le statut affiche **"En attente..."** jusqu'au démarrage d'une simulation.



Interface initiale — 9 places disponibles, en attente de démarrage

6. Modes de synchronisation

L'application propose deux mécanismes de synchronisation pour contrôler l'accès concurrent aux places :

6.1 Mode MUTEX

Cliquez sur le bouton ■ **MUTEX** (vert) pour démarrer la simulation en mode Mutex. Un mutex (MUTual EXclusion) garantit qu'une seule voiture à la fois peut accéder à la section critique de gestion des places. Les voitures s'exécutent en séquence pour l'accès au parking, évitant tout conflit.

6.2 Mode SÉMAPHORE

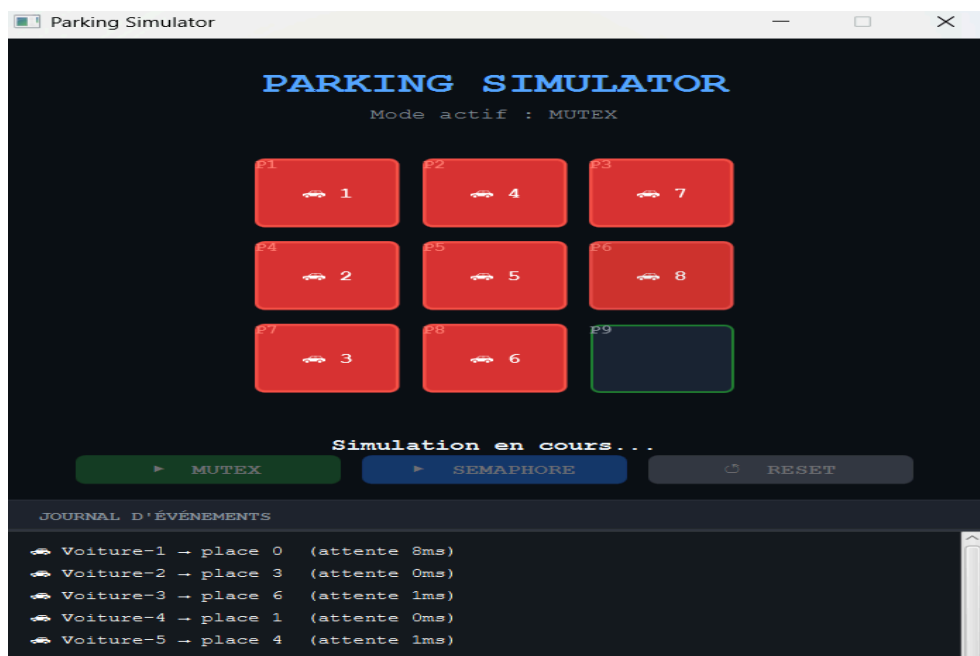
Cliquez sur le bouton ■ **SÉMAPHORE** (bleu) pour démarrer en mode Sémaphore. Un sémaphore compteur autorise jusqu'à N voitures simultanément (N = nombre de places libres). Ce mode est généralement plus performant car il permet plus de parallélisme.

Critère	MUTEX	SÉMAPHORE
Parallélisme	Faible (accès exclusif)	Élevé (N accès simultanés)
Attente	Active (spin-lock possible)	Passive (blocage + signal)
Utilisation CPU	Plus élevée	Plus faible
Simplicité	Simple à implémenter	Légèrement plus complexe

7. Déroulement d'une simulation

7.1 Simulation en cours

Pendant la simulation, les places occupées s'affichent en **rouge** avec le numéro de la voiture garée. La place encore disponible reste avec bordure verte. Le journal d'événements en bas de fenêtre liste en temps réel les arrivées (Voiture-N → place X) avec leur temps d'attente.



Simulation en cours — 8 places occupées (rouge), 1 libre (verte)

7.2 Fin de simulation

Lorsque toutes les voitures ont quitté le parking, le message "✓ **Simulation terminée !**" s'affiche. Toutes les places redeviennent disponibles (fond sombre). Le bouton **RESET** devient actif pour relancer une nouvelle simulation.



Simulation terminée — toutes les places libérées, bouton RESET actif

8. Journal d'événements (parking.log)

Chaque simulation génère automatiquement un fichier **logs/parking.log**. Ce fichier contient l'historique complet de la simulation avec horodatage précis.

Format d'une entrée de log :

```
[2026-05-14 20:39:22] Voiture-1 | GAREE | Place 0 | Attente : 0 ms
[2026-05-14 20:39:24] Voiture-1 | SORTIE | Quitte la place 0
```

Champs du journal :

Champ	Description
[timestamp]	Date et heure précises de l'événement
Voiture-N	Identifiant de la voiture (thread)
GAREE / SORTIE	Action : stationnement réussi ou départ
Place X	Numéro de la place utilisée (0 à 8)
Attente : X ms	Temps attendu avant de trouver une place libre

```
≡ parking.log M X
SysAdm > logs > ≡ parking.log
1 [2026-05-14 20:39:22] SIMULATION | Mode : MUTEX | Places : 9 | Voitures : 0
2 [2026-05-14 20:39:22] -----
3 [2026-05-14 20:39:22] Voiture-1 | GAREE | Place 0 | Attente : 0 ms
4 [2026-05-14 20:39:22] Voiture-2 | GAREE | Place 3 | Attente : 0 ms
5 [2026-05-14 20:39:22] Voiture-3 | GAREE | Place 6 | Attente : 0 ms
6 [2026-05-14 20:39:22] Voiture-4 | GAREE | Place 1 | Attente : 0 ms
7 [2026-05-14 20:39:23] Voiture-5 | GAREE | Place 4 | Attente : 1 ms
8 [2026-05-14 20:39:23] Voiture-6 | GAREE | Place 7 | Attente : 0 ms
9 [2026-05-14 20:39:23] Voiture-7 | GAREE | Place 2 | Attente : 0 ms
10 [2026-05-14 20:39:24] Voiture-8 | GAREE | Place 5 | Attente : 0 ms
11 [2026-05-14 20:39:24] Voiture-9 | GAREE | Place 8 | Attente : 0 ms
12 [2026-05-14 20:39:24] Voiture-1 | SORTIE | Quitte la place 0
13 [2026-05-14 20:39:24] Voiture-10 | GAREE | Place 0 | Attente : 0 ms
14 [2026-05-14 20:39:25] Voiture-2 | SORTIE | Quitte la place 3
15 [2026-05-14 20:39:25] Voiture-11 | GAREE | Place 3 | Attente : 643 ms
16 [2026-05-14 20:39:26] Voiture-4 | SORTIE | Quitte la place 1
17 [2026-05-14 20:39:26] Voiture-13 | GAREE | Place 1 | Attente : 747 ms
18 [2026-05-14 20:39:26] Voiture-3 | SORTIE | Quitte la place 6
19 [2026-05-14 20:39:26] Voiture-14 | GAREE | Place 6 | Attente : 489 ms
20 [2026-05-14 20:39:27] Voiture-10 | SORTIE | Quitte la place 0
21 [2026-05-14 20:39:27] Voiture-15 | GAREE | Place 0 | Attente : 813 ms
22 [2026-05-14 20:39:27] Voiture-6 | SORTIE | Quitte la place 7
23 [2026-05-14 20:39:27] Voiture-12 | GAREE | Place 7 | Attente : 2209 ms
24 [2026-05-14 20:39:28] Voiture-5 | SORTIE | Quitte la place 4
25 [2026-05-14 20:39:28] Voiture-8 | SORTIE | Quitte la place 5
26 [2026-05-14 20:39:28] Voiture-13 | SORTIE | Quitte la place 1
27 [2026-05-14 20:39:28] Voiture-7 | SORTIE | Quitte la place 2
28 [2026-05-14 20:39:29] Voiture-9 | SORTIE | Quitte la place 8
29 [2026-05-14 20:39:29] Voiture-15 | SORTIE | Quitte la place 0
30 [2026-05-14 20:39:30] Voiture-12 | SORTIE | Quitte la place 7
31 [2026-05-14 20:39:30] Voiture-14 | SORTIE | Quitte la place 6
32 [2026-05-14 20:39:30] Voiture-11 | SORTIE | Quitte la place 3
33
```

Journal parking.log — Simulation complète en mode MUTEX (32 événements)

Le journal ci-dessus montre une simulation MUTEX avec 9 places et 15 voitures. Les 9 premières voitures se garent instantanément (Attente : 0 ms). Les voitures suivantes attendent jusqu'à 2209 ms qu'une place se libère.

9. Réinitialisation (Reset)

Après la fin d'une simulation, cliquez sur le bouton ■ **RESET** pour remettre le parking à l'état initial. Toutes les places redeviennent libres, le journal est vidé, et vous pouvez relancer une nouvelle simulation en mode MUTEX ou SÉMAPHORE.

10. Résolution de problèmes fréquents

Problème : mvn : commande introuvable

Solution : Maven n'est pas dans le PATH. Ajoutez C:\Program Files\apache-maven-3.9.15\bin dans les variables d'environnement système

Problème : java : version incorrecte

Solution : Vérifiez que JAVA_HOME pointe vers JDK 21. Exécutez java -version pour confirmer.

Problème : BUILD FAILURE à la compilation

Solution : Exécutez mvn clean compile -X pour voir les erreurs détaillées. Vérifiez que pom.xml est bien présent.

Problème : Fenêtre JavaFX ne s'ouvre pas

Solution : Vérifiez que JavaFX est bien configuré dans pom.xml (openjfx 21.0.1). Essayez mvn clean package puis relancez.

Problème : Fichier parking.log vide

Solution : Lancez la simulation complètement (jusqu'à 'Simulation terminée'). Le log n'est écrit qu'à la fin de chaque événement.

11. Résumé des commandes

Commande	Description
<code>git clone <url></code>	Cloner le dépôt pour la première fois
<code>git pull</code>	Synchroniser avec les dernières modifications
<code>mvn clean compile</code>	Nettoyer et recompiler le projet
<code>mvn javafx:run</code>	Lancer l'interface graphique
<code>mvn clean package</code>	Créer le JAR exécutable dans target/
<code>mvn test</code>	Lancer les tests unitaires